

#route.tcl

Description : Global + detailed routing and post-route optimization.

Tool : Synopsys IC Compiler II (ICC2) or similar

Flow order :

1. Open \${design}_post_cts_done

2. Set routing layers and options

3. Global route

4. Detail route

5. Post-route optimization (timing / DRC / antenna)

6. Save as \${design}_route_done

#

Key Variables (from user_setup.tcl):

max_core - Max CPU cores

ndm_lib_name - Library name under db/

design - Top-level design name

min_route_layer - Min routing layer

max_route_layer - Max routing layer

enable_ccd_route - Enable CCD during route_opt (0/1)

route_effort - Route effort (low/medium/high) – optional

stop_after_route - If 1, stop after routing stage

Read Technology & User Settings

source -e -v ./scripts/tech_setup.tcl

source -e -v ./scripts/user_setup.tcl

Multi-CPU Setting

```
set_host_options -max_cores $max_core
```

```
# Open NDM Library and Create Route Block
```

```
if { [file exists "db/${ndm_lib_name}"] } {  
    puts "\n[INFO] Library 'db/${ndm_lib_name}' exists, opening...\n"  
    open_lib db/${ndm_lib_name}  
    copy_block -from ${design}_post_cts_done -to ${design}_route_done  
    current_block ${design}_route_done  
    link  
} else {  
    puts "\n[ERROR] NDM library 'db/${ndm_lib_name}' does not exist.  
Please check.\n"  
    return  
}
```

```
# Routing Layer Settings
```

```
puts "\n[INFO] Setting routing layer range for routing: $min_route_layer ..  
$max_route_layer\n"
```

```
set_ignored_layers \  
    -min_routing_layer $min_route_layer \  
    -max_routing_layer $max_route_layer
```

```
# Route App Options
```

```
puts "\n[INFO] Setting route/route_opt application options.\n"
```

```
# General options
```

```
set_app_options -name route.global.effort    -value high
```

```
set_app_options -name route.detail.effort      -value high
set_app_options -name route.common.connect_pg_nets -value true
set_app_options -name route.common.preserve_clock_routes -value true
```

Post-route optimization

```
set_app_options -name route_opt.flow.enable      -value true
set_app_options -name route_opt.flow.eco_mode    -value false
set_app_options -name route_opt.flow.driven_by   -value timing
```

Optional: user-controlled effort override

```
if { [info exists route_effort] && $route_effort ne "" } {
    puts "[INFO] Overriding routing effort to '$route_effort'"
    set_app_options -name route.global.effort -value $route_effort
    set_app_options -name route.detail.effort -value $route_effort
}
```

CCD during route_opt

```
if { [info exists enable_ccd_route] && $enable_ccd_route } {
    puts "\n[INFO] CCD enabled during post-route timing optimization.\n"
    set_app_options -name route_opt.flow.enable_ccd -value true
} else {
    puts "\n[INFO] CCD disabled during post-route timing optimization.\n"
    set_app_options -name route_opt.flow.enable_ccd -value false
}
```

Connect PG Nets Before Routing

```
puts "\n[INFO] Ensuring PG connectivity before routing.\n"
```

```
connect_pg_net -automatic
```

```
# Global Routing
```

```
puts "\n[INFO] Starting global routing.\n"
```

```
route_global
```

```
# Optional: global route-based congestion analysis can be added here.
```

```
# Detailed Routing
```

```
puts "\n[INFO] Starting detailed routing.\n"
```

```
route_detail
```

```
# Post-Route Optimization (Timing / DRC / Antenna)
```

```
puts "\n[INFO] Running post-route optimization (timing/DRC/antenna).\n"
```

```
route_opt -from final_route -to final_route
```

```
# If you want a separate ECO step later, you can save an intermediate block here:
```

```
# save_block -as ${design}_route_opt_done
```

```
# Basic Signoff Checks – Example (optional)
```

```
puts "\n[INFO] Optional: running basic route checks (shorts/open, min spacing, etc.).\n"
```

```
check_routes
```

```
# Additional commands (uncomment if you use them in your flow):
```

```
# check_drc
```

```
# check_antenna
```

```
# Save Block After Routing
```

```
puts "\n[INFO] Saving block after routing.\n"
```

```
connect_pg_net -automatic
```

```
save_block -as ${design}_route_done
```

```
# Exit or Stop After Route
```

```
sh touch route_done
```

```
if { $stop_after_route }{
```

```
    puts "\n[INFO] Route stage completed (stop_after_route = 1).  
    Returning...\n"
```

```
    return
```

```
} else {
```

```
    puts "\n[INFO] Route stage completed. Closing design and exiting  
    tool.\n"
```

```
    close_blocks -force
```

```
    close_lib
```

```
    exit
```

```
}
```